

Cours 2 — Conditions et boucles

Semaine 2 ■ Manuel Hachette ch. 2 (§ 2.3, 2.4) ■ 1 h

Objectifs

- Écrire et évaluer une expression booléenne
- Utiliser `if` / `elif` / `else`
- Répéter des instructions avec `while` et `for ... in range(...)`

1. Expressions booléennes

Une expression booléenne vaut `True` ou `False`.

Comparaisons : `==` (égal), `!=` (différent), `<`, `<=`, `>`, `>=`

Ne pas confondre `=` (affectation) et `==` (test d'égalité).

Opérateurs logiques : `and`, `or`, `not`

```
age = 17
(age >= 15) and (age < 18)      # True
not (age == 18)                 # True
```

2. L'instruction conditionnelle

```
if note >= 16:
    print("Très bien")
elif note >= 10:
    print("Admis")
else:
    print("Non admis")
```

- La condition est suivie de `:` ; le **bloc** exécuté est **indenté** (4 espaces).
- `elif` et `else` sont facultatifs ; les conditions sont testées **dans l'ordre**, un seul bloc est exécuté.
- L'indentation n'est pas décorative : c'est elle qui délimite les blocs en Python.

3. La boucle conditionnelle `while`

« Tant que la condition est vraie, répéter le bloc. »

```
n = 1
while n <= 100:
    print(n)
    n = n * 2          # 1, 2, 4, 8, 16, 32, 64
```

Si la condition ne devient jamais fausse, le programme **boucle à l'infini**. Il faut qu'une variable de la condition soit modifiée dans le corps de la boucle.

On utilise `while` quand on **ne sait pas d'avance** combien de répétitions il faudra (attendre une bonne réponse, atteindre un seuil...).

4. La boucle bornée `for`

« Pour chaque valeur de la séquence, répéter le bloc. »

```
for i in range(5):           # i prend les valeurs 0, 1, 2, 3, 4
    print(i)
```

- `range(n)` : de 0 à $n - 1$ (**n valeurs**, n exclu).
- `range(a, b)` : de a à $b - 1$.
- `range(a, b, p)` : de a à $b - 1$ par pas de p.

Accumulateur, un motif à connaître par cœur :

```
total = 0
for i in range(1, 11):
    total = total + i        # somme 1 + 2 + ... + 10
print(total)                # 55
```

À retenir

- `==` teste, `=` affecte.
- Un bloc `= :` puis indentation.
- `for` quand on connaît le nombre de tours, `while` sinon.
- `range(n)` commence à 0 et s'arrête à $n - 1$.
- Motif accumulateur : initialiser avant la boucle, mettre à jour dedans.